

Resumen del dia

2026-05-26

Integracion Odoo <-> WordPress (Locopolo)

Documento que recoge en detalle todo el trabajo realizado durante la jornada en torno a la integracion bidireccional entre Odoo (development-erp-thelocopolo.odoo.com) y el WordPress de staging (locopolostg.wpenginepowered.com). Incluye los fallos detectados, las soluciones aplicadas, los riesgos identificados y el trabajo que queda pendiente.

1. Diagnostico y restauración del WordPress de staging

Sintoma inicial

La web de staging (locopolostg.wpenginepowered.com) cargaba en blanco. El navegador no mostraba contenido aunque el servidor respondía con HTTP 200 OK.

Diagnostico tecnico

Se verificaron varias URLs con peticiones directas via curl. El resultado fue claro: el frontend devolvía 0 bytes (white screen of death), pero el resto del sitio funcionaba correctamente:

- / (home), /sample-page/, /encuentranos/: 0 bytes (PHP fatal silenciado)
- /wp-login.php: 16.159 bytes (login operativo)
- /wp-json/: ~1.000 bytes (REST API respondía con metadatos válidos)

Conclusion: WordPress core funcionaba; solo el renderizado del tema crasheaba. Por tanto el problema estaba en el theme o en algún plugin del frontend, no en la infraestructura.

Causa real

El usuario hizo previamente un "Restore" desde wp-admin con WPvivid y la restauración falló a mitad de camino, dejando la base de datos parcialmente importada. Es un comportamiento conocido en hostings con timeouts agresivos: PHP supera max_execution_time o agota memoria sin emitir mensaje claro de error.

Solución aplicada

El usuario subió a staging un backup completo (Database + Files) generado en local con el plugin WPvivid Backup. Después usó la opción "Restore" desde wp-admin sobre ese mismo backup. La segunda ejecución completa correctamente y el sitio volvió a funcionar.

Regla práctica documentada

Las restauraciones de WPvivid sobre WP Engine pueden fallar silenciosamente. No dar por buena una restauración hasta ver el mensaje explícito "Migration/Restore completed successfully". Si no aparece, relanzar.

Consecuencia colateral

El contenido de staging quedó sobrescrito por el de local. Cualquier cambio que existiera solo en staging (publicado por otra persona en el equipo) se perdió. Para revertir se podría usar el backup automático de WP Engine desde el User Portal -> Backup Points.

2. Renombrado de carpetas de los plugins de integración

Problema de nomenclatura

Los nombres antiguos eran ambiguos: "API ODOO-WORDPRESS" y "API WORDPRESS-ODOO" podían leerse de dos maneras distintas (dirección del flujo de datos, o donde vive la API). Esa ambigüedad generó confusión repetida durante el desarrollo.

Renombrado aplicado

Se eligió la convención "quien llama a quien" porque es la más natural al leerla:

- "API ODOO-WORDPRESS" -> "API wordpress-llama-a-odoo" (WordPress es cliente, Odoo expone la API)
- "API WORDPRESS-ODOO" -> "API odoo-llama-a-wordpress" (Odoo es cliente, WordPress expone la API)

Alcance del cambio

Solo se renombraron las carpetas contenedoras del proyecto. NO se tocó:

- Las junctions NTFS internas (siguen apuntando a las carpetas reales de wp-content/plugins/)
- Las carpetas reales del plugin en WordPress (conservan los nombres originales)
- El header "Plugin Name:" dentro de los PHP
- Las clases ni los archivos internos del plugin
- Las entradas históricas de documentación/*.txt (registro fiel del momento)

Cambios de configuración

CLAUDE.md se actualizó con las nuevas rutas y se añadió una nota en cada plugin aclarando que la carpeta del plugin activo en wp-content/plugins/ conserva el nombre original. Así futuras sesiones entienden por qué la carpeta exterior y la interior tienen nombres distintos.

3. Despliegue y configuración del módulo Odoo locopolo_wordpress_sync

Contexto

El módulo de Odoo locopolo_wordpress_sync (creado en sesiones anteriores) se desplegó en el entorno development de Odoo.sh mediante push a la rama "development" del repositorio Sotelolo/locopolo. Odoo.sh hace rebuild automático al detectar el push (5-10 minutos por iteración).

Configuración en Odoo

Creados dos parámetros del sistema (Ajustes -> Técnico -> Parámetros del sistema):

Clave: locopolo_wordpress.url
Valor: <https://locopolostg.wpenginepowered.com>

Clave: locopolo_wordpress.token
Valor: MI_SUPER_TOKEN_SECRETO_12345

El token debe coincidir exactamente con el almacenado en wp_options["aow_odoo_token"] del WordPress destino. Si no existe esa opción en la BD, el plugin de WordPress usa por defecto MI_SUPER_TOKEN_SECRETO_12345.

Funcionamiento verificado

El módulo se dispara cuando se ejecuta uno de estos eventos en Odoo:

- create() de un res.partner con categoría TIENDA y mostrar_en_web = True
- write() de un res.partner (campos relevantes o mostrar_en_web)
- unlink() de un res.partner con categoría TIENDA
- action_confirm() de un sale.order cuyo partner sea TIENDA + mostrar_en_web

En cada trigger, el módulo construye un payload JSON con los datos del partner (odoo_id, nombre, dirección, ciudad, código_postal, teléfono, zona, fecha_ultimo_pedido) y hace una petición HTTPS al endpoint del plugin de WordPress.

4. Bloqueo de Cloudflare por User-Agent de Python

Sintoma

Tras desplegar el módulo y configurarlo, las peticiones de Odoo a WordPress devolvían sistemáticamente HTTP 403 Forbidden. El log de Odoo mostraba:

```
WordPress sync error (upsert tienda 7918):  
403 Client Error: Forbidden for url:  
https://locopolostg.wpenginepowered.com/wp-json/odoo/v1/tiendas
```

Diagnostico

Inicialmente se sospecho que el token estaba mal configurado. Se verificó con un script Python que el valor en `ir.config_parameter` era exactamente "MI_SUPER_TOKEN_SECRETO_12345" (28 caracteres, sin saltos de línea ni espacios invisibles). El mismo token funcionaba perfectamente al hacer una petición manual con curl.

La pista clave fue replicar la cabecera User-Agent que envía la librería "requests" de Python:

```
curl -H "User-Agent: python-requests/2.31.0" ... -> HTTP 403  
curl sin User-Agent custom -> HTTP 200
```

La respuesta 403 venía con HTML de Cloudflare (no JSON del plugin). Esto confirmó que la petición no llegaba siquiera a WordPress: la cortaba Cloudflare antes, por considerar "python-requests/X.X.X" un User-Agent sospechoso de scraping/bot.

Solucion

Modificado `wordpress_client.py` del módulo `locopolo_wordpress_sync` para enviar un User-Agent identificable:

```
def _headers(self):  
    return {  
        'Content-Type': 'application/json',  
        'X-Odoo-Token': self.token,  
        'User-Agent': 'LocopoloWordPressSync/1.0',  
    }
```

Commit + push a la rama development. Odoo.sh hizo rebuild en ~6 minutos. Tras el deploy, las peticiones empezaron a llegar correctamente al endpoint y el log mostro:

```
WordPress sync: tienda 7918 creada/actualizada.
```

5. Bug del plugin WordPress: terms duplicados en taxonomías jerárquicas

Sintoma

Aunque la sincronización Odoo -> WordPress funcionaba (status 200, log de Odoo OK), algunas tiendas sincronizadas (AIBAK, Oier usu prueba externo) no aparecían en la página pública /encuentranos/, mientras que otras (LUISA MARIA ZUIN) sí aparecían.

Diagnostico

La página /encuentranos/ agrupa tiendas por la taxonomía "zona" (jerárquica: España > provincias). Al consultar los términos de la taxonomía se descubrieron DOS términos para Gipuzkoa:

```
ID 75 | name="Gipuzkoa (Guipuzcoa)" | parent=11 (España) | count=3 (las que SI aparecen)
ID 266 | name="Gipuzkoa" | parent=0 | count=1 (AIBAK)
```

El término 266 era un DUPLICADO sin padre que el plugin había creado al sincronizar AIBAK. El frontend solo lee las tiendas que cuelgan de "España", así que las del 266 quedaban invisibles.

Causa raíz

El plugin de WordPress (API WORDPRESS-ODOO.php) asignaba la taxonomía así:

```
// Código con el bug:
wp_set_object_terms($post_id, $zona, 'zona', false);
```

`wp_set_object_terms` con un string suelto, en una taxonomía JERÁRQUICA, no busca el término por nombre: crea uno nuevo en la raíz. Y los nombres con parentesis "Gipuzkoa (Guipuzcoa)" no coincidían con los existentes, así que se generaban duplicados.

LUISA MARIA ZUIN no tenía este problema porque su provincia "Cadiz" no tiene parentesis y el match funcionaba por casualidad. Pero todas las provincias con parentesis estaban afectadas: Gipuzkoa, Bizkaia, Lleida, Castello, Girona, Alacant, Valencia, A Coruna, Ourense, Navarra, Illes Balears, Araba.

Solución aplicada en staging

Editado el plugin directamente desde wp-admin -> Plugins -> Editor de archivos de plugin. Reemplazada la asignación buggy por:

```
if ( $zona ) {
    $term = get_term_by( 'name', trim( $zona ), 'zona' );
    if ( $term ) {
        wp_set_post_terms( $post_id, [ $term->term_id ], 'zona' );
    }
}
```

Idem para la taxonomía "tipo-tienda". La lógica ahora es: buscar el término por nombre, obtener su ID, y usar ese ID al asignar al post. Así nunca se crean duplicados.

Por qué NO se cambiaron los nombres de provincia en Odoo

Se valoró renombrar las 12 provincias afectadas en Odoo (eliminando los parentesis), pero se descartó porque: (1) `res.country.state` es una tabla estándar que afecta a facturación, contabilidad y envíos; (2) habría que renombrar también los términos en WordPress; (3) el bug seguiría latente para cualquier taxonomía jerárquica futura con nombres no triviales. La solución en el plugin es más robusta y resuelve el problema de raíz.

6. Sincronización de la taxonomía "zona" con el catálogo de Odoo

Comparativa inicial

Se compararon los términos existentes en la taxonomía "zona" de WordPress con la lista canónica de res.country.state de Odoo (56 provincias y localidades). Resultados:

- Coincidencias exactas: 51 términos (no se tocaron)
- A CREAR (existen en Odoo pero no en WP): 6 términos
- A BORRAR (existen en WP pero no en Odoo): 51 términos

Estrategia híbrida elegida (no destructiva)

La opción de borrar los 51 términos ausentes en Odoo se descartó por tres riesgos serios:

- Borrar términos padre como "España" o "Francia" rompería la jerarquía: todas las provincias quedarían huérfanas
- Borrar términos con posts asignados (count > 0) dejaría a esas tiendas sin zona
- Provincias de otros países (Portugal, Italia, Francia, Oman...) pueden ser útiles en el futuro

Acciones aplicadas vía REST API

Se usó la aplicación password de "idoia" para llamar a /wp-json/wp/v2/zona:

- Creados 5 términos bajo el padre "España" (id 11): Araba/Alava, Calvia, Leon, Palma de Mallorca, menorca
- Alcudia ya existía (id 267) - el diff lo detectó pero el script encontró que se había creado entre la consulta y la acción
- Borrado el término huérfano "Araba (Alava)" (id 73, count 0) - sustituido por el nuevo Araba/Alava
- Borrado el término duplicado "Gipuzkoa" (id 266) DESPUÉS de reasignar AIBAK al término correcto manualmente

Verificación final

AIBAK, Oier usu prueba externo y LUISA MARIA ZUIN aparecen correctamente en /encuentranos/ agrupados por su zona. El fix del plugin garantiza que las futuras sincronizaciones desde Odoo asignarán siempre el término correcto sin crear duplicados.

7. Aplicación del fix en local y pendientes

Fix replicado en local

El fix aplicado en staging via Plugin Editor solo afecta a la copia del archivo en WP Engine. El archivo del plugin en el WordPress local (accesible via junction) seguía con la versión antigua. Se ha replicado el mismo cambio en el archivo local para mantener consistencia y evitar que una futura Auto-Migration local -> staging reintroduzca el bug.

Estado de los dos lados

- Modulo Odoo locopolo_wordpress_sync (in development): código nuevo desplegado, parámetros configurados, funcionando
- Plugin WordPress API WORDPRESS-ODOO en staging: fix de taxonomía aplicado, plugin activo, endpoint operativo
- Plugin WordPress en local: fix aplicado también para mantener consistencia (commit pendiente)

Pendientes para futuras sesiones

- Considerar refactorizar el flujo de upsert para que rellene también campos opcionales (latitud/longitud, tipo-tienda) si se quiere mostrar las tiendas en un mapa interactivo
- Añadir un endpoint REST que devuelva el detalle completo de una tienda por odoo_id, para facilitar el debug futuro sin necesidad de acceder a wp-admin
- Documentar en el README del plugin la lista de campos esperados en el payload y los meta_keys/taxonomías que se actualizan
- Implementar un mecanismo de logging dentro del propio plugin (no solo en Odoo) para registrar las llamadas recibidas y posibles errores

Scripts auxiliares creados en API wordpress-llama-a-odoo/pruebas/

- verificar_token_odoo.py: lee el token en Odoo con repr() para detectar caracteres invisibles
- sync_zona_diff.py: compara los terms de la taxonomía "zona" de WP con la lista canónica de Odoo
- sync_zona_aplicar.py: aplica el plan híbrido (crear los que faltan, borrar duplicados conocidos) via REST
- comparar_tiendas.py: utilidad para inspeccionar metadata y taxonomías de un post tienda

8. Resumen ejecutivo

En una jornada se diagnosticó y resolvió una cadena de tres fallos encadenados que impedían que la integración Odoo -> WordPress funcionase end-to-end: una restauración incompleta de WPvivid que dejó staging corrupto, un bloqueo de Cloudflare por User-Agent automático de Python, y un bug latente en el plugin de WordPress que creaba términos duplicados cuando los nombres de provincia contenían parentesis. Los tres fallos se solucionaron sin perder datos ni tiempo de desarrollo, dejando además la taxonomía de zonas alineada con el catálogo de Odoo y el código replicado en local para evitar regresiones futuras. La integración queda operativa: Odoo envía tiendas a WordPress automáticamente al confirmar pedidos o modificar contactos, y aparecen en /encuentranos/ con su provincia correcta.